

Programação Pareada em um Casca de Noz

Entenda de forma simples e direta as vantagens e armadilhas do pareamento

Chamada

Programação pareada é uma prática simples de aprender, pouco custosa de instaurar e traz diversas vantagens para o projeto e o time. Nesse artigo vamos mergulhar fundo -- muito além das explicações superficiais que frequentemente recebemos quando somos apresentados ao pareamento. Ofereceremos ao leitor que já parecia um embasamento sólido para tornar seu pareamento mais agradável e produtivo. Ao que tentou e não gostou, uma perspectiva do que pode ter atrapalhado e sugestões de como ter experiências melhores. E, se você nunca ouviu falar sobre programação pareada, aproveite para se atualizar!

Fotos a colocar pelo artigo (fotos gerais, podem ser usadas em qualquer lugar)

<https://dl.dropboxusercontent.com/u/9092538/dois-pares.png>

<https://dl.dropboxusercontent.com/u/9092538/par-noturno.png>

<https://dl.dropboxusercontent.com/u/9092538/pareamento-editoracao.png>

<https://dl.dropboxusercontent.com/u/9092538/tres-pares.png>

Introdução

A essa altura, quase todos nós já ouvimos falar de programação pareada, vários de nós experimentaram a técnica e um número crescente de pessoas a praticam diariamente. Por essa razão, pode soar um tanto estranho escrever um artigo apenas sobre programação pareada. O fato é que, embora a ideia central do pareamento seja bastante difundida, é comum encontrarmos pessoas que não tiram tanto proveito quanto poderiam da técnica e até mesmo vários que contam casos de terror ao tentar aplicá-lo: de quedas incríveis de produtividade a uma extrema sensação de frustração em pareamentos. Talvez pela simplicidade da prática e das vantagens óbvias, nos deixamos ficar com a impressão de que não precisávamos estudá-la mais a fundo, ignorando potenciais ganhos secundários e caindo em armadilhas um tanto previsíveis.

Justamente para combater essa ideia de que programação pareada é simplesmente duas pessoas trabalhando no mesmo computador, discutiremos, ao longo deste artigo, por que a programação pareada é produtiva e como ela pode não ser produtiva. Em quais situações devemos parear? Como parear? Em quais podemos abrir mão dela? Quem deveria parear com quem? Esperamos que essas e outras questões sejam bem respondidas por este artigo.

Tópico 1. O que é programação pareada e o que eu ganho com isso?

Muito brevemente, programação pareada é uma **técnica de desenvolvimento na qual dois desenvolvedores trabalham juntos sobre um mesmo problema e em uma mesma máquina**. Isso é a base, mas a prática não para aí! Esses desenvolvedores trabalham colaborativamente, conversando intensamente sobre o problema e buscando uma boa solução para ele.

Costumamos chamar o desenvolvedor que está no controle do teclado de "piloto"; o desenvolvedor ao lado é conhecido como "co-piloto". Como mencionado acima, a ideia é que ambos troquem informações sobre o problema que estão a resolver e implementem juntos a solução final.

Duas pessoas com pontos de vista diferentes sobre o código é interessante: o piloto geralmente está muito envolvido com problemas da linguagem (abrir e fechar chaves, sintaxe, semântica), enquanto o co-piloto está enxergando tudo em um nível mais abstrato.

Mas quais as vantagens? Muitas. Uma delas é simplicidade: como ambos os desenvolvedores estão discutindo sobre a solução do problema, é de se esperar que eles lancem ideias primárias que, então, evoluem em possibilidades melhores e que eles finalmente concordem com uma solução mais simples e elegante.

A qualidade do código então sai ganhando: além do fato de que não precisamos defender as vantagens de um código simples, o código é escrito por dois desenvolvedores juntos. Se um desenvolvedor escrever código que o outro não entende, isso será rapidamente corrigido. Por consequência, esse código terá sido lido por ao menos 2 desenvolvedores e a chance dele ser legível por outros desenvolvedores da equipe é ainda maior.

Ainda em relação à qualidade de código, é possível argumentar que desenvolvedores que praticam programação pareada fazem menos uso de "gambiarras técnicas". A pressão que o colega ao lado exerce, de forma indireta e não-proposital, faz com que o desenvolvedor pense duas vezes antes de escrever um código de menor qualidade. Esse é, afinal, um comportamento do ser humano: ninguém gosta de mostrar ao colega do lado que "sabe menos".

Um outro efeito indireto da pressão do colega é foco. Ao trabalhar sozinho, é comum que o desenvolvedor perca o foco por diferentes motivos. Hoje em dia, com a quantidade de novas informações que temos ao nosso dispor nas mais variadas redes sociais, manter o foco é uma arte. Mas, com um colega ao lado, é mais difícil para o desenvolvedor abrir sua rede social favorita ou ver suas mensagens privadas com alguém ao lado. Mesmo distrações de trabalho são evitadas: as pessoas pensam duas vezes antes de interromper duas pessoas que estão trabalhando juntas. A consequência disso tudo é um maior foco na atividade e, portanto, um aumento de produtividade.

Aprendizado também é outro fruto indireto da programação pareada. Ao parear, ambos os desenvolvedores aprendem um com outro. O conhecimento sobre regras de negócio ou mesmo o conhecimento bem técnico, que é conhecido somente por um dos desenvolvedores, no momento do pareamento, é transferido para o outro desenvolvedor. Isso faz com que não existam mais especialistas (ou "pais") de certos trechos do software, diminuindo o chamado *fator do caminhão* (ver box).

Dados os pontos mencionados acima, é bastante claro que programação pareada só tem a agregar para o processo de desenvolvimento de uma equipe. O grande porém é que parear não é tão fácil quanto parece.

Box: "Fator Caminhão" (ou Fator Trator) *é o nome dado, em eXtreme Programming, para o problema de quando apenas algumas pessoas conseguem mexer em determinados trechos de código. O nome vem da brincadeira: "quantos desenvolvedores um caminhão precisa atropelar até que seu projeto pare?". Uma piada de mau gosto, mas que faz sentido. O conhecimento não pode ficar centralizado nas mãos de um único desenvolvedor.*

Tópico 2. Como parear com qualidade?

Programar pareado é uma prática que transcende o lado técnico: o lado social é bastante utilizado. E todos nós sabemos que o lado social não é geralmente o mais aguçado em desenvolvedores de software.

Um caso típico é quando o piloto pega o teclado e sai escrevendo código sem puxar qualquer discussão com o co-piloto. Nesses casos, estamos desperdiçando o talento de um desenvolvedor, pois o co-piloto fica entediado e não consegue cooperar com o piloto.

Programar pareado é então mais do que sentar ao lado de outro desenvolvedor. Ser piloto é mais do que apenas escrever linhas de código, e ser co-piloto é mais do que apenas sentar ao lado. Um sinal de que o pareamento não está indo bem é quando vemos duas pessoas em silêncio, apenas olhando para o monitor, ou mesmo quando o co-piloto apenas consente com tudo que o piloto diz.

Durante a sessão de pareamento, espera-se que o piloto e o co-piloto discutam e reflitam sobre o problema usando diversas formas de comunicação. Ambos podem conversar, rabiscar, escrever provas de conceito. É tarefa de ambos puxar o colega ao lado para conversar.

O piloto também tem suas obrigações específicas. Ele deve explicar o código que está escrevendo, tanto para diminuir a ansiedade do co-piloto quanto para ter chance de receber feedback sobre ele. Já o co-piloto também deve entender que o piloto tem o seu jeito de programar e deve palpitar em momentos oportunos. Por exemplo, o co-piloto não precisa avisar o piloto que ele esqueceu o ponto-e-vírgula no fim da linha no momento em que isso aconteceu, já que o piloto provavelmente já percebeu o erro, mas só optou por escrever a linha debaixo para não perder o raciocínio.

A dupla também deve perceber quem será o melhor piloto e o melhor co-piloto naquele momento. Não há problema algum em revezar; isso é bem visto, inclusive, e discutimos mais abaixo. Se o co-piloto está falando mais do que o piloto naquele momento (porque provavelmente conseguiu entender melhor o problema atual), talvez ele seja um melhor piloto do que o atual. Uma maneira rápida e fácil de perceber isso é perceber quando o co-piloto está ditando código para o piloto.

Box: Foco absoluto

Em nossas aulas de agilidade, quando pedimos para os alunos trabalharem em par, eles ficam tão concentrados que nem percebem que os instrutores fazem malabarismos na frente da sala. Todos ficam extremamente surpresos quando o instrutor comenta, ao final do exercício. Isso mostra como as pessoas ficam focadas quando estão pareando.

Tópico 3. Discussões comuns

Há uma grande mística com relação a verdades e lendas do pareamento. Em eXtreme Programming, sugere-se que os desenvolvedores de um time passem muito do tempo deles, perto dos cem por cento, programando em pares. Há também quem acredite que programação pareada custa caro para o projeto, já que, enquanto poderiam trabalhar em duas tarefas concorrentemente, dois desenvolvedores do time trabalham sobre uma tarefa só.

Vejamos as discussões mais comuns sobre a prática com um pouco mais de profundidade.

Sub-tópico 3.1. Parear 100% do tempo?

Falamos cedo sobre as vantagens do pareamento no foco, na simplicidade e design do código. Logo, é fácil imaginar que o desenvolvedor deveria parear 100% do seu tempo. Embora seja uma recomendação de XP, há pouquíssimos times no mundo que conseguem de fato programar pareado cem por cento do tempo de trabalho.

Parear todo o tempo é exaustivo para a maior parte das pessoas, já que a capacidade de se manter focado de cada um de nós costuma ser bastante limitada. Não é a toa que uma hora-aula tenha a duração real de 50 minutos: há diversos estudos que indicam que pessoas têm, na média, uma capacidade de manter foco limitada a 40 minutos por vez, e então elas precisam de uma pausa.

Em uma jornada normal de trabalho, ficar exausto muito cedo é uma má ideia. Quando perceber cansaço, lembre-se que não é preciso parear até terminar de uma tarefa -- a todo momento, qualquer pessoa do par pode pedir uma pausa para recarregar as energias. Não esqueça também que cada pessoa tem o seu limite máximo de foco. Respeite o limite do seu par; se ele cansou, descansa ou eventualmente gaste o resto da sua energia para trabalhar em tarefas mais simples, que não exijam pareamento.

Sub-tópico 3.2. Falta de privacidade

Além da dificuldade de manter foco por longos períodos, há também outra questão que pode causar incômodos. Programar pareado envolve mais interação com a pessoa que está sentada do seu lado e menos com o resto do mundo. Isso pode ser um desafio, a princípio, já que você provavelmente não trabalha com seus melhores amigos.

Com uma pessoa ao seu lado, olhando para a sua tela, não temos tantas chances de verificar e-mails, ou chats, ou mensagens. E, em um mundo em que tendemos, por trabalho ou lazer, a divagar por abas do navegador e ferramentas sociais a cada poucos minutos, parear pode causar a sensação de perda de privacidade no mundo virtual. E essa sensação não se limita ao virtual, já que programar pareado ainda envolve estar fisicamente mais perto do seu par.

E, de fato, esses são pontos de privacidade dos quais abrimos mão quando programamos em par, tanto porque não queremos que o colega veja mensagens pessoais nossas, quanto em respeito ao nosso par, que abriu mão até da máquina dele para se sentar com você.

É claro que ninguém espera que você passe o dia sem se distrair. Apenas, enquanto estiver pareando, faça tudo o que puder para permanecer focado -- e quando precisar desfocar, pare de parear.

Box: Etiqueta no pareamento

Quando parear, demonstre consideração pelo seu par:

- 1. Desligue avisos sonoros de chat; se possível, desligue os messengers em si;*
- 2. Tire (ambos) os fones de ouvido; foque em escutar seu par;*
- 3. Higiene pessoal é fundamental. Não esqueça de escovar os dentes e do desodorante;*
- 4. Quando for co-piloto, não roube o teclado do colega, fale com ele;*
- 5. Não dite código. Se perceber que ajudaria mais pilotando, peça para trocar;*
- 6. Pareie com todos da equipe, não apenas com seus amigos. Leve a troca de conhecimento como parte do trabalho.*

Sub-tópico 3.3. Parear em todas as tarefas

Ainda mais uma questão relacionada a parear cem por cento do tempo é que, entendendo que os principais benefícios da programação pareada são manter um padrão de código único no projeto, a simplicidade e a troca de conhecimento entre membros do time, nota-se que esses ganhos estão presentes quando o pareamento acontece para resolver problemas complexos, mas são bem menores quando tratamos tarefas simples.

Um exemplo disso é, talvez, a criação de um CRUD, um sisteminha simples apenas para inserir, listar, alterar e remover dados: por envolver tarefas muito básicas, o código tende mesmo a sair simples e padronizado. Além disso, como é um código que todo desenvolvedor sabe fazer sem pensar, o aprendizado é bem limitado.

Outra atividade em que geralmente não faz sentido trabalhar em par é uma atividade de estudo. Estudar é parte do dia a dia de todo desenvolvedor. É comum pararmos de programar para entrar no Google e entender melhor como determinada API funciona. Se a busca inicial for rápida, faça em par. Mas, se a atividade de pesquisa for complicada e demandar trabalho, prefira fazer isso separado. Cada pessoa tem sua velocidade de leitura e de interpretação e fazer isso em par pode não ser produtivo. Portanto, nessas situações, estude separado e, quando encontrar a solução, volte a trabalhar em par.

Parear em tarefas bobas também pode ser bastante desmotivante, especialmente para o co-piloto. Embora haja times que realmente preferem parear todo o tempo, nós frequentemente preferimos não parear em tarefas muito corriqueiras.

Sub-tópico 3.4. Trabalhar com mais de 2 desenvolvedores juntos?

A ideia da programação pareada é ter dois desenvolvedores trabalhando juntos para resolver um problema. No entanto, algumas equipes extrapolam esse número e fazem com que 3 ou até 4 desenvolvedores trabalhem juntos para resolver o problema. A ideia pode fazer algum sentido já que, se com 2 pessoas o código fica bom, com 3 ficaria melhor ainda. O problema é que, na prática, não é isso que acontece. É difícil manter um canal de comunicação entre os vários desenvolvedores ao mesmo tempo e um deles acaba sendo sub-utilizado.

Uma boa prática é então nunca fazer qualquer variação do pareamento, como um "trireamento". Nos casos onde a equipe tem 3 desenvolvedores, o terceiro desenvolvedor pode, no momento em que está sozinho, realizar atividades de revisão de código, resolver bugs, trabalhar em histórias mais simples ou mesmo estudar. Claro, rotacione os pares com frequência para que esse desenvolvedor não trabalhe sozinho o dia todo.

Box: Casos em que faz sentido "trirear"

Há situações em que pode ser vantajoso para a equipe toda trabalhar junta numa tarefa. Por exemplo: construir a base de código do projeto, ou um trecho de código que usa um conhecimento muito específico e é muito importante para o projeto. Nesses casos, pode ser importante a participação da equipe toda. No entanto, a dinâmica costuma ser diferente: o piloto costuma ter o comando da situação, fazendo uma espécie de apresentação para toda a equipe.

Sub-tópico 3.5. Quando trocar de par?

Falando em rotacionar pares, outra questão frequente é por quanto tempo um par deve trabalhar junto -- e, nesse aspecto, não há uma resposta certa. Há times que escolhem um par para o dia, outros que preferem variar mais, trocando de pares a cada poucas horas, e até os que formam pares que duram até terminarem uma tarefa.

Enquanto trabalhamos com o mesmo par mantemos a linha de raciocínio e o ritmo de trabalho e, se uma troca de pares acontecer, será necessário reduzir o ritmo e explicar o que está sendo feito para o novo par. Isto é, de fato existe um gasto de tempo para explicar o que está acontecendo. Esse gasto pode ser considerado perda de tempo. Ou também é possível encarar essa re-explicação como uma oportunidade de verificar se o código está simples o bastante, mesmo para quem o vê pela primeira vez. Ganha-se de um lado, perde-se de outro.

Apenas é preciso atentar para que os pares específicos não se repitam demais. Seja por afinidade pessoal ou por se sentirem mais à vontade com uma mesma parte do código, tome cuidado para não parear sempre com as mesmas pessoas. Um membro do time deveria criar o hábito de parear com todos os outros para evitar criar ilhas de conhecimento. Uma forma boa de descobrir esses pares muito recorrentes é adicionar um quadro de pareamento às suas métricas.

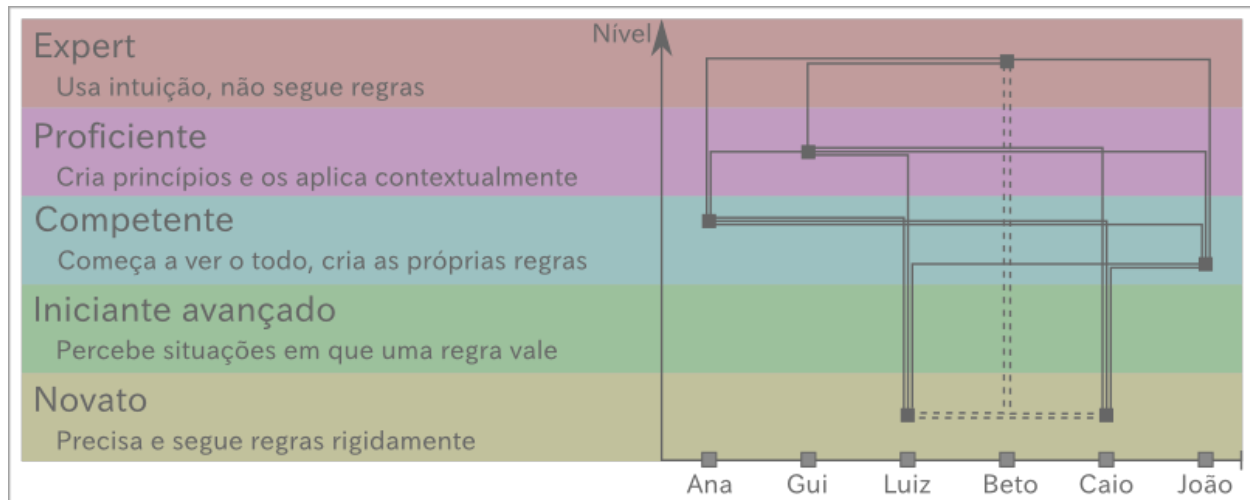


FOTO: <https://dl.dropboxusercontent.com/u/9092538/quadro-pareamento.png>

Sub-tópico 3.6. Quais níveis de conhecimento ganham mais

Finalmente, uma das principais reclamações de times que começam a praticar a programação pareada são aquelas sessões em que os dois desenvolvedores saem exaustos e frustrados. Frequentemente porque um não entendeu nada do que aconteceu e o outro porque não conseguiu se expressar bem o bastante.

O que acontece é que pessoas com graus muito diferentes de conhecimento podem ter dificuldades em acompanhar o raciocínio do outro. Isso é um fenômeno bem conhecido na área que estuda modelos de aprendizado. Veja a ilustração do modelo de aprendizagem de Dreyfus na figura 1.



SVG da imagem em: <https://dl.dropboxusercontent.com/u/9092538/dreyfus-pair.svg>

O que acontece no pareamento é que os estágios de Dreyfus costumam parear bem com níveis próximos e os níveis intermediários costumam parear bem com outras pessoas de mesmo nível. Os maiores problemas costumam acontecer em duas situações: primeiro, quando dois novatos pareiam e não têm quem dê regras claras para eles seguirem, é comum que eles se sintam perdidos e travem; segundo, quando um novato pareia com um *expert*, já que o primeiro precisa de regras e o segundo já possui um conhecimento tão enraizado do negócio que ele não segue mais regras -- é mais como se um *expert* seguisse sua intuição para decidir o que fazer. Nesse cenário, é comum que o novato não consiga entender o raciocínio do expert e que esse não o consiga explicar, porque para ele as variáveis que o fizeram tomar as decisões de código são tantas e o processo de escolha tão rápido que a sensação é de que aquilo é, ou óbvio, ou muito difícil de explicar.

Em suma, evite as ligações tracejadas na figura 1. Lembre-se também que programação pareada não é *coaching*. Se o desenvolvedor for realmente bem iniciante, talvez valha a pena fazer com que ele passe por uma sessão de *coaching*, para depois integrá-lo à equipe.

Outra questão frequente nisso é sobre o pareamento entre pessoas de mesmo nível. Por serem consideradas de uma mesma categoria, podemos ter a impressão de que conhecimentos semelhantes não vão dar um ganho grande para time e projeto. Apenas, então, lembre-se que duas pessoas da mesma categoria não necessariamente chegaram nela pelo mesmo caminho. Além disso, podem pertencer ao mesmo grupo mas, ainda assim, conhecerem coisas diferentes e aprenderem bastante uns com os outros. Pareamento dentro do mesmo nível, exceto para novatos, vale sim a pena.

Tópico 4. Como introduzir PP em uma equipe?

Programar pareado não é fácil. Como mencionamos acima, é uma atividade social, onde duas pessoas se comunicam o tempo inteiro. Existem diversos desafios no momento em que uma equipe decide parear.

Um primeiro desafio é a timidez. Estima-se que por volta de 75% das pessoas sente algum nível de nervosismo na hora de se apresentar em público. De maneira similar, não é fácil programar com alguém te observando. É tarefa do mais experiente da dupla de facilitar a conversa e "quebrar o gelo" com o menos experiente -- e, com certeza, o que está mais nervoso com a situação.

Além disso, rotacionar os pares com frequência pode ser benéfico no começo. Assim, nenhum desenvolvedor fica estressado, todos trabalham com todos, e a experiência torna-se, além de produtiva, mais divertida.

Se seu gerente ou algum outro membro da equipe, por alguma razão, não é fã da prática, comece devagar. Escolha um projeto menor para piloto e pareie. Vá experimentando a prática, e descobrindo se ela se encaixa em seu contexto ou não. Mas não desista dela, afinal, listamos aqui muitos motivos para que você a pratique.

Tópico 5. O que se fala de pareamento por aí?

Apesar da prática de programação pareada ter se tornado popular após o livro de XP do Kent Beck, ela é com certeza praticada muito antes de a conhecermos pelo seu nome. O próprio Frederick Brooks (autor do famosíssimo livro *The Mythical Man-Month*) uma vez comentou que ele, quando aluno de graduação entre 1953 e 1956, escreveu 1500 linhas de código totalmente livres de defeito na primeira vez que praticou programação pareada.

Programação pareada talvez seja a prática ágil na qual a ciência mais realizou estudos a respeito. Uma ótima referência sobre o assunto é o capítulo sobre Programação Pareada do livro "Making Software: What Really Works, and Why We Believe It", editado por Andy Oram e Greg Wilson. Muitos estudos foram feitos com estudantes e profissionais da indústria em projetos de pequeno e grande porte.

A pesquisadora americana Laurie Williams mostrou que estudantes que parearam ao longo da atividade proposta escreveram códigos que passavam em 15% mais testes do que estudantes que trabalham sozinhos. Isso significa que pessoas que programam pareado escrevem código com menos bug do que aqueles que optam por trabalhar de maneira individual.

Já na indústria, muitas equipes reportaram a diversos pesquisadores uma grande melhoria da qualidade do produto. A troca de conhecimento, bem como a maior legibilidade dos códigos escritos em par também é algo que aparece constantemente nesses estudos.

Em relação à produtividade, que é sempre um assunto polêmico em qualquer prática de engenharia de software, os estudos devem ser lidos com cautela. Programação pareada é considerada produtiva quando a tarefa desempenhada é complexa. Alguns estudos mostram o contrário: uma queda na produtividade. No entanto, diversos pesquisadores já criticaram esses estudos, pois eles foram feitos em projetos pequenos com tarefas pequenas.

De forma resumida então, é possível concluir que estudos em programação pareada, em sua grande maioria, mostram que o uso da prática é benéfico. Se você quiser os detalhes desses estudos, leia o o capítulo do livro *Making Software*, citado anteriormente, que é um excelente resumo deles todos.

Tópico 6. Programação pareada e produtividade

Produtividade é um assunto sempre muito polêmico em engenharia de software. Uma das razões para tal é que é muito difícil medir produtividade de um desenvolvedor. A maneira mais comum de fazer isso é medir quantas linhas de código um desenvolvedor/equipe escreve por dia. Mas qualquer um com o mínimo de

experiência em desenvolvimento de software sabe que a quantidade de linhas que um desenvolvedor escreve não diz nada sobre sua produtividade.

Sempre buscamos por práticas que aumentam a produtividade da equipe e fugimos das que a diminuem. Uma prática produtiva é aquela que faz, entre outras coisas, com que o código produzido funcione conforme o esperado e seja fácil de ser mantido a longo prazo.

Programação pareada é uma daquelas práticas que, se não for analisada com atenção, dá a sensação de que a equipe está perdendo produtividade, afinal, são duas pessoas resolvendo um único problema. Se olharmos a curto prazo, talvez a sensação de perda de produtividade seja natural. Porém, a médio prazo, a programação pareada mostra suas vantagens, pois o código produzido contém menos bugs, gerando menos retrabalho para a equipe. Além disso, a troca de conhecimento entre os colegas é outro fruto difícil de mensurar, mas especialmente útil em equipes que trabalham em projetos complexos ou legados. Enfim, tudo o que foi discutido acima, a longo prazo, se destaca e gera benefícios.

Obviamente, como discutido anteriormente, existem momentos ideais para se parear, e momentos nos quais a prática não fará muita diferença. A curva de aprendizado também não é simples. Programar pareado não é fácil; a quebra de paradigma no começo é complicada. Mas equipes maduras, que já pareiam há algum tempo, têm ganhos de produtividade.

Tópico 7. Desafios em aberto

Muitos dos mitos relacionados a prática foram combatidos nesse artigo, mas ainda há muito o que estudar, aprender e melhorar a respeito de programação pareada.

Combater o hábito, por exemplo, é um deles. Desenvolvedores que estão começando com programação pareada podem ter a sensação de que trabalhar "com alguém mais devagar" não é produtivo, ou que não conseguem pensar com alguém ao lado. É por isso que equipes de software devem pensar bem sobre como introduzir a prática entre seus desenvolvedores.

Discussões econômicas também são polêmicas. Ainda é difícil para um gerente ou, em nível maior, uma empresa, não pensar que duas pessoas estão sendo "gastas" no lugar de uma só. Novamente a dificuldade de se enxergar a médio prazo atrapalhando a equipe de fazer uso de boas práticas. O exemplo disso é justamente a frase do gerente da Microsoft, que diz que prefere pagar um desenvolvedor acima da média do que pagar dois desenvolvedores medianos. São argumentos difíceis de serem combatidos, mas esperamos que esse artigo tenha mostrado que, a médio e longo prazo, os benefícios da programação pareada superam os gastos com ambos os desenvolvedores.

Outro grande desafio é a execução de programação pareada em equipes distribuídas. Existem diversas técnicas para equipes que não trabalham colocadas, mas neste artigo não discutimos esse tópico com profundidade.

Tópico 8. Conclusão

Programação pareada é com certeza uma atividade benéfica para equipes que desenvolvem software. Neste artigo citamos diferentes vantagens que a prática agrega ao processo, como simplicidade, troca de

conhecimento, maior qualidade (tanto interna quanto externa) do código, entre outras.

Mas também tentamos deixar claro aqui que programar pareado não é fácil. Quebrar a barreira da timidez, saber o momento de parear (e de parar de parear), revezar os pares, etc é desafiador. Equipes que não sabem lidar bem com programação pareada podem não extrair todos os benefícios e, em casos piores, até perderem produtividade e motivação. Fique atento aos detalhes e lembre-se de perguntar para seu time quais problemas eles têm sentido e mostrar para eles que é possível aprender a aproveitar melhor o pareamento.

Talvez um ponto que não mencionamos ao longo deste artigo é que programar pareado é divertido e ajuda a construir uma sensação de time. Além de produzir código de qualidade, você se diverte e dá risada com o colega ao lado. A parte social é algo que frequentemente esquecemos de comentar, mas que tem uma alta importância em equipes que produzem software -- e métodos ágeis falam bastante disso.

Portanto, escreva código de qualidade e seja feliz. Como? Praticando programação em par.

Tópico 9. Referências

Beck, Kent. Extreme Programming Explained: Embrace Change. Addison-Wesley Professional, Segunda Edição, 2004.

Fowler, Martin. Pair Programming Misconceptions.

<http://martinfowler.com/bliki/PairProgrammingMisconceptions.html>

Oram, Andy; Wilson, Greg. Making Software: What Really Works and Why We Believe it. O'Reilly Media, Primeira Edição, 2010.

Tópico 9. Autores

Mauricio Aniche atua pela Caelum como instrutor e desenvolvedor. É mestre em Ciência da Computação e atual aluno de doutorado pela Universidade de São Paulo. Mauricio é apaixonado por engenharia de software, e foca seus estudos em Test-Driven Development e qualidade de código. É autor do livro "Test-Driven Development: Teste e Design no Mundo Real", publicado pela Casa do Código. Ultimamente é muito apaixonado por métricas de código, e lançou recentemente a ferramenta Codesheriff.com.

Foto:

https://lh3.googleusercontent.com/-MINK_V0ShHE/AAAAAAAAAAI/AAAAAAAAADSs/IDK3n1qdl8I/photo.jpg

Cecilia Fernandes pode ser encontrada desenvolvendo software, dando aulas e atuando como Agile coach na Caelum. Ela ainda é presença frequente em eventos focados em agilidade tanto como palestrante, quanto como voluntária, revisora de submissões e, na Agile Brazil, como organizadora. Ela acredita em melhoria contínua e foca seus esforços na criação de bons times, pessoal e tecnicamente, e na cultura de crescimento e inovação.

Foto: <https://dl.dropboxusercontent.com/u/9092538/ceci-fernandes.png>